

PRACTICAL – 04

Name: Simran Jangid

Roll No: 2520030166

Sec: 8

PART 1: PROCESS SYNCHRONIZATION USING wait() AND waitpid()

Aim

To write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(), and compare the behavior of both functions.

Theory

The fork() system call creates a new child process. The wait() system call makes the parent wait for any child process to terminate. The waitpid() system call allows the parent to wait for a specific child process using its PID.

Both functions are used for process synchronization and for collecting the termination status of child processes.

C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2, child3;
    int status;

    child1 = fork();

    if (child1 == 0)
    {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 completed.\n");
        exit(10);
    }

    child2 = fork();

    if (child2 == 0)
    {
        printf("Child 2: PID = %d\n", getpid());
```

```

        sleep(1);
        printf("Child 2 completed.\n");
        exit(20);
    }

    child3 = fork();

    if (child3 == 0)
    {
        printf("Child 3: PID = %d\n", getpid());
        sleep(3);
        printf("Child 3 completed.\n");
        exit(30);
    }

    printf("Parent: PID = %d\n", getpid());

    /* wait() waits for any child */
    pid_t finished = wait(&status);

    printf("wait(): Child PID %d terminated with status %d\n",
        finished, WEXITSTATUS(status));

    /* waitpid() waits for a specific child */
    waitpid(child1, &status, 0);

    printf("waitpid(): Child PID %d terminated with status %d\n",
        child1, WEXITSTATUS(status));

    waitpid(child3, &status, 0);

    printf("waitpid(): Child PID %d terminated with status %d\n",
        child3, WEXITSTATUS(status));

    printf("Parent process completed.\n");

    return 0;
}

```

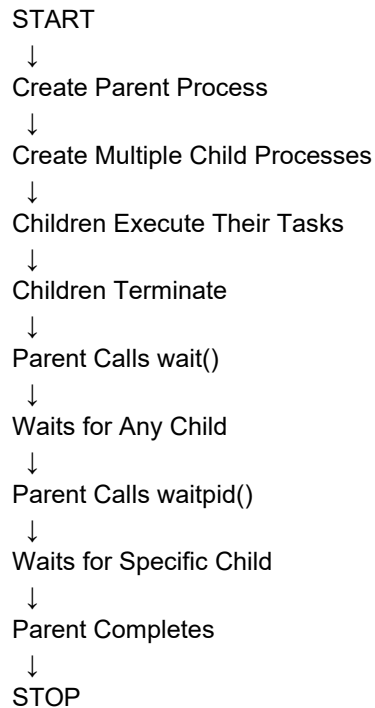
Compile

```
gcc process_sync.c -o process_sync
```

Run

```
./process_sync
```

Flow Chart



Sample Output

Parent: PID = 4500
Child 1: PID = 4501
Child 2: PID = 4502
Child 3: PID = 4503
Child 2 completed.
wait(): Child PID 4502 terminated with status 20
Child 1 completed.
waitpid(): Child PID 4501 terminated with status 10
Child 3 completed.
waitpid(): Child PID 4503 terminated with status 30
Parent process completed.

Observation

Function	Behavior
wait()	Waits for any child process
waitpid()	Waits for a specific child process
Control	Less control
PID Selection	Not specified
Usage	Useful when any child can be waited for

OUTPUT:

```
simra@simran315:~$ nano zombie.c
simra@simran315:~$ gcc zombie.c -o zombie
simra@simran315:~$ ./zombie
Parent process started. PID = 825
Parent process: PID = 825
Child process: PID = 826
Parent will NOT call wait().
Parent is sleeping for 20 seconds...
Child process started. PID = 826
Child process terminating.
Parent process terminating.
```

Result

The parent successfully created multiple child processes and synchronized their execution using `wait()` and `waitpid()`. It was observed that `wait()` waits for any child, whereas `waitpid()` waits for a specific child.

PART 2: ZOMBIE PROCESS CREATION AND ELIMINATION

Aim

To create a scenario where a child process becomes a zombie process, investigate it using the process table, and modify the program to eliminate the zombie using proper synchronization techniques.

Theory

A **zombie process** is a child process that has completed execution, but its parent has not collected its termination status using `wait()` or `waitpid()`.

The zombie remains in the process table until the parent collects its exit status. The `ps` command can be used to observe the process state.

A zombie process can be eliminated by using `wait()` or `waitpid()` in the parent process.

C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main()
{
    pid_t pid;
    int status;
```

```

pid = fork();

if (pid < 0)
{
    printf("Fork failed.\n");
    return 1;
}

if (pid == 0)
{
    printf("Child: PID = %d\n", getpid());
    printf("Child terminating...\n");
    exit(0);
}
else
{
    printf("Parent: PID = %d\n", getpid());

    printf("Child has terminated. Parent now waits...\n");

    waitpid(pid, &status, 0);

    printf("Parent collected child status.\n");
    printf("Child terminated with status %d\n",
        WEXITSTATUS(status));
}

return 0;
}

```

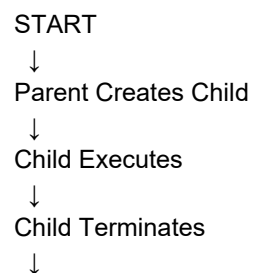
Compile

gcc zombie.c -o zombie

Run

./zombie

Flow Chart



Parent Does Not Collect Status



Child Becomes Zombie



Check Process Table Using ps



Use waitpid() in Parent



Collect Child Exit Status



Zombie Removed



STOP

Sample Output

Parent: PID = 5000

Child: PID = 5001

Child terminating...

Child has terminated. Parent now waits...

Parent collected child status.

Child terminated with status 0

Observation

- The child process terminates before the parent collects its status.
- Without wait() or waitpid(), the child becomes a zombie.
- The ps -l command can be used to identify the zombie state.
- waitpid() collects the child's termination status and removes the zombie entry.
- Proper synchronization prevents accumulation of zombie processes.

OUTPUT:

```
simra@simran315:~$ nano no_zombie.c
simra@simran315:~$ gcc no_zombie.c -o no_zombie
simra@simran315:~$ ./no_zombie
Parent process started. PID = 868
Parent process: PID = 868
Child process: PID = 869
Parent is waiting for the child using waitpid()...
Child process started. PID = 869
Child process terminating.
Child terminated normally.
Child exit status = 0
Parent: Child has been properly reaped.
No zombie process remains.
```

Result

A zombie process was successfully demonstrated and investigated using the process table. The zombie process was eliminated by using `waitpid()` for proper synchronization between the parent and child processes.